```
!nvidia-smi
```

```
Fri Oct 16 05:58:07 2026
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version:
+-----------------------------------------+------------------------+-----------
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Un
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  C
|                                         |                        |
|=========================================+========================+===========
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |
| N/A   39C    P8              9W /   70W |       0MiB /  15360MiB |      0%
|                                         |                        |
+-----------------------------------------+------------------------+-----------

+-----------------------------------------------------------------------------+
| Processes:
|  GPU   GI   CI              PID   Type   Process name                      G
|        ID   ID                                                            U
|=============================================================================
|  No running processes found
+-----------------------------------------------------------------------------
```

```
!nvidia-smi --query-gpu=name,compute_cap --format=csv
```

```
name, compute_cap
Tesla T4, 7.5
```

```
%%writefile ex1_timed.cu
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>
#include <sys/time.h>

#define N 512

// Runs on the GPU: one addition per block
__global__ void add(int *a, int *b, int *c) {
    int i = blockIdx.x;
    c[i] = a[i] + b[i];
}

// Fill an array with random integers from 0 to 99
void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

// Read the CPU wall clock in seconds
```

```
double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

int main(void) {
    int *a, *b, *c;           // Arrays in CPU memory
    int *d_a, *d_b, *d_c;     // Arrays in GPU memory
    int size = N * sizeof(int);
    double dStart, dElaps;

    // Allocate GPU memory
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Allocate CPU memory and prepare inputs
    a = (int *)malloc(size);
    b = (int *)malloc(size);
    c = (int *)malloc(size);

    random_ints(a, N);
    random_ints(b, N);

    // Copy inputs from CPU to GPU
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Start timing
    dStart = cpuSecond();

    add<<<N, 1>>>(d_a, d_b, d_c);
    cudaDeviceSynchronize();

    // Stop timing
    dElaps = cpuSecond() - dStart;

    // Copy results from GPU to CPU
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    // Display the first five results
    for (int r = 0; r < 5; r++)
        printf("%d + %d = %d\n", a[r], b[r], c[r]);

    // Check every result
    int errors = 0;
    for (int r = 0; r < N; r++) {
        if (c[r] != a[r] + b[r])
            errors++;
    }
```

```
    printf("Verification: %d mismatches out of %d\n", errors, N);
    printf("Time taken - %.7f seconds\n", dElaps);

    // Release memory
    free(a);
    free(b);
    free(c);
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);

    return 0;
}
```

```
Overwriting ex1_timed.cu
```

```
!nvcc -arch=sm_75 ex1_timed.cu -o ex1_timed && ./ex1_timed
```

```
83 + 77 = 160
86 + 34 = 120
77 + 90 = 167
15 + 26 = 41
93 + 24 = 117
Verification: 0 mismatches out of 512
Time taken - 0.0001001 seconds
```

```
%%writefile ex2_check.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define N 512

// Checks CUDA runtime calls and reports errors.
#define CHECK(call)                                                    \
    do {                                                               \
        const cudaError_t error = (call);                              \
        if (error != cudaSuccess) {                                    \
            printf("Error: %s:%d, ", __FILE__, __LINE__);              \
            printf("code:%d, reason: %s\n", error,                     \
                   cudaGetErrorString(error));                         \
            exit(1);                                                   \
        }                                                              \
    } while (0)

__global__ void add(int *a, int *b, int *c) {
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
```

```
void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

int main(void) {
    int *a, *b, *c;
    int *d_a, *d_b, *d_c;
    size_t size = N * sizeof(int);
    double dStart, dElaps;

    // TODO 1: Allocate GPU memory
    CHECK(cudaMalloc((void **)&d_a, size));
    CHECK(cudaMalloc((void **)&d_b, size));
    CHECK(cudaMalloc((void **)&d_c, size));

    a = (int *)malloc(size);
    random_ints(a, N);

    b = (int *)malloc(size);
    random_ints(b, N);

    c = (int *)malloc(size);

    // TODO 2: Copy data from CPU to GPU
    CHECK(cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice));

    dStart = cpuSecond();

    // Launch GPU kernel
    add<<<N, 1>>>(d_a, d_b, d_c);

    // TODO 3: Check kernel launch and execution errors
    CHECK(cudaGetLastError());
    CHECK(cudaDeviceSynchronize());

    dElaps = cpuSecond() - dStart;

    // TODO 4: Copy result from GPU to CPU
    CHECK(cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost));

    for (int r = 0; r < 5; r++)
        printf("%d + %d = %d\n", a[r], b[r], c[r]);
```

```
    // Verify results
    int errors = 0;
    for (int r = 0; r < N; r++)
        if (c[r] != a[r] + b[r])
            errors++;

    printf("Verification: %d mismatches out of %d\n", errors, N);
    printf("Time taken - %.7f\n", dElaps);

    free(a);
    free(b);
    free(c);

    // TODO 5: Free GPU memory
    CHECK(cudaFree(d_a));
    CHECK(cudaFree(d_b));
    CHECK(cudaFree(d_c));

    return 0;
}
```

```
Writing ex2_check.cu
```

```
!nvcc -arch=sm_75 ex2_check.cu -o ex2_check && ./ex2_check
```

```
83 + 77 = 160
86 + 34 = 120
77 + 90 = 167
15 + 26 = 41
93 + 24 = 117
Verification: 0 mismatches out of 512
Time taken - 0.0005372
```

```
%%writefile ex3_picture.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define WIDTH 800
#define HEIGHT 600

#define CHECK(call)                                                    \
    do {                                                               \
        const cudaError_t error = (call);                              \
        if (error != cudaSuccess) {                                    \
            printf("Error: %s:%d, ", __FILE__, __LINE__);              \
            printf("code:%d, reason: %s\n", error,                     \
                   cudaGetErrorString(error));                         \
            exit(1);                                                   \
```

```
        }                                                                    \
    } while (0)

double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

// PictureKernel: doubles each pixel value, with a maximum of 255.
__global__ void PictureKernel(unsigned char *d_Pin,
                              unsigned char *d_Pout,
                              int height, int width) {

    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    if ((Row < height) && (Col < width)) {
        int v = 2 * d_Pin[Row * width + Col];
        d_Pout[Row * width + Col] = (v > 255) ? 255 : v;
    }
}

int main(void) {
    size_t count = (size_t)WIDTH * HEIGHT;
    size_t size = count * sizeof(unsigned char);

    unsigned char *h_in, *h_out;
    unsigned char *d_in, *d_out;

    double dStart, dElaps;

    h_in = (unsigned char *)malloc(size);
    h_out = (unsigned char *)malloc(size);

    CHECK(cudaMalloc((void **)&d_in, size));
    CHECK(cudaMalloc((void **)&d_out, size));

    // TODO 1: Generate random pixel values from 0 to 255.
    for (size_t i = 0; i < count; i++) {
        h_in[i] = rand() % 256;
    }

    CHECK(cudaMemcpy(d_in, h_in, size, cudaMemcpyHostToDevice));

    // TODO 2: Define 16 x 16 threads per block.
    dim3 threadsPerBlock(16, 16);

    // TODO 3: Calculate the number of blocks, rounding up.
    dim3 numBlocks((WIDTH + 15) / 16,
                   (HEIGHT + 15) / 16);
```

```
printf("Grid: %d x %d blocks of %d x %d threads\n",
       numBlocks.x, numBlocks.y,
       threadsPerBlock.x, threadsPerBlock.y);

dStart = cpuSecond();

// TODO 4: Launch the kernel.
PictureKernel<<<numBlocks, threadsPerBlock>>>(
    d_in, d_out, HEIGHT, WIDTH
);

CHECK(cudaGetLastError());
CHECK(cudaDeviceSynchronize());

dElaps = cpuSecond() - dStart;

CHECK(cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost));

// TODO 5: Display the original top-left 50 x 50 pixels.
printf("\nOriginal Picture (50 x 50):\n");

for (int row = 0; row < 50; row++) {
    for (int col = 0; col < 50; col++) {
        printf("%3d ", h_in[row * WIDTH + col]);
    }
    printf("\n");
}

// Display the updated top-left 50 x 50 pixels.
printf("\nUpdated Picture (50 x 50):\n");

for (int row = 0; row < 50; row++) {
    for (int col = 0; col < 50; col++) {
        printf("%3d ", h_out[row * WIDTH + col]);
    }
    printf("\n");
}

// TODO 6: Verify every pixel and count capped pixels.
int errors = 0;
int capped = 0;

for (size_t i = 0; i < count; i++) {
    int expected = 2 * h_in[i];

    if (expected > 255) {
        expected = 255;
        capped++;
    }
```

```
            if (h_out[i] != expected) {
                errors++;
            }
        }

        printf("\nVerification: %d mismatches out of %zu\n",
               errors, count);

        printf("Pixels capped at 255: %d\n", capped);
        printf("Time taken - %.7f\n", dElaps);

        free(h_in);
        free(h_out);

        CHECK(cudaFree(d_in));
        CHECK(cudaFree(d_out));

        return 0;
    }
```

```
Writing ex3_picture.cu
```

```
!nvcc -arch=sm_75 ex3_picture.cu -o ex3_picture && ./ex3_picture
```

```
255 255  58 210 114 255  12 206 255 108 255   4 255 255  34 255 104  48 255  9
122 255 255  78 255 255  34 106 120  74  12 255 255 255  70  40 255  72 186 25
114 102 180 108 255 255 120 178 255 228 255 255 255 146 228 255 255 255 255 10
255 252  76 255  34  12 255 136 128 255 100 255 255 255 255 255 255 255  70 21
255 255 146 222 255 150 250  62 255  44 255 255 255  32 255 210 255 255 255 16
 30  96  54  74 140  76 166  74 194  18 255 255   2 255  36 204 255 255 255 25
255 255  40 255 255 210 255 192 234  44 142 214 255 224 162 255 255 255  70 16
255 255 255 255 146 255 208  42 222 255  20 182 255   4 100 255 255 255  58 23
 34 255 182 255  94 255 180 255 255 255 255 146 255 255 180 255 210 124 255 13
255  18 200 144 255  12 255 206 255 120 255 255  54  82 255 255 255 204  90 22
255 240 176 255 255 255 230 255 255 255 255 218 138  74 230 255  72 255  42 25
 56  82 255 255  36 255 128 158   2  74 255   0 110 106 255 120 255 255 118 25
255 150 255  50 255 194 255 255 124  90 255 255 255  34 150 255 255 255  60 25
 84 255 255 255 255 255 255  98 255 176 255  20  98  58 158 255 255 255 248  3
106 114 255 255 255 138 255 255 236 138 178 200  66 174 255 124 255  72 255 25
255 174 128 255 255 255 255 255 255 244 255 255 255 255  64 206 255 255 255  2
 20 255  76  64 255 255 255 255 196 255 255 150 255 162 255  92  88 255 255 25
255 230  14  52 255  90 255 255 255   6 234 164 255 255 255 124 255  30 132 23
255 255 255 255 255 255 255 255 255 255 134 255 255 255 154 255   6  20 255 16
238 255 255 255 255  38 255 255 255 108  86 255 250 255  92 255  58 232 255 19
255  82 255 130 255 255  80 255 208 172 255 255 255 255 255 255 255 124  50 25
255 238  50  86 255   8 255 182  44 255 255 218 255 255   0  80 255  76 126 13
255 255 255 255 168 255 222  30 255 150  62 108 148 224 204 255 255  36 134  8
254 100 170 255 255 178 255 255 255 154 255 192 184 255 255 255 255 104 255  2
 24 255 204 255  80  20 108  66  68 255 140 252 255 196 255 255 255 240 255 25

Verification: 0 mismatches out of 480000
Pixels capped at 255: 240121
Time taken - 0.0001330
```

```
%%writefile ex4_volume.cu
#include <cstdio>
#include <cstdlib>
#include <sys/time.h>

#define NX 500
#define NY 500
#define NZ 500

#define CHECK(call)                                                    \
    do {                                                               \
        const cudaError_t error = (call);                              \
        if (error != cudaSuccess) {                                    \
            printf("Error: %s:%d, ", __FILE__, __LINE__);              \
            printf("code:%d, reason: %s\n", error,                     \
                   cudaGetErrorString(error));                         \
            exit(1);                                                   \
        }                                                              \
    } while (0)

double cpuSecond() {
    struct timeval tp;
```

```
    gettimeofday(&tp, NULL);
    return (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6;
}

// Element-wise multiplication of 3D arrays.
__global__ void mulElem3D(int *a, int *b, int *c,
                          int nx, int ny, int nz) {

    // TODO 1: Calculate x, y and z coordinates.
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int z = blockIdx.z * blockDim.z + threadIdx.z;

    // TODO 2: Check all three dimensions.
    if (x < nx && y < ny && z < nz) {

        // TODO 3: Calculate the flat index.
        size_t idx = (z * ny + y) * nx + x;

        c[idx] = a[idx] * b[idx];
    }
}

void random_ints(int *x, size_t size) {
    for (size_t i = 0; i < size; i++) {
        x[i] = rand() % 100;
    }
}

int main(void) {

    size_t count = (size_t)NX * NY * NZ;
    size_t size = count * sizeof(int);

    int *a, *b, *c;
    int *d_a, *d_b, *d_c;

    double dStart, dElaps;

    // Allocate GPU memory.
    CHECK(cudaMalloc((void **)&d_a, size));
    CHECK(cudaMalloc((void **)&d_b, size));
    CHECK(cudaMalloc((void **)&d_c, size));

    // Allocate CPU memory and generate values.
    a = (int *)malloc(size);
    b = (int *)malloc(size);
    c = (int *)malloc(size);

    random_ints(a, count);
    random_ints(b, count);
```

```
// Copy input arrays to GPU.
CHECK(cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice));
CHECK(cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice));

// 8 x 8 x 8 = 512 threads per block.
dim3 threadsPerBlock(8, 8, 8);

// TODO 4: Calculate grid dimensions, rounding up.
dim3 numBlocks(
    (NX + 7) / 8,
    (NY + 7) / 8,
    (NZ + 7) / 8
);

printf("Grid: %d x %d x %d blocks\n",
       numBlocks.x, numBlocks.y, numBlocks.z);

dStart = cpuSecond();

// TODO 5: Launch the 3D kernel.
mulElem3D<<<numBlocks, threadsPerBlock>>>(
    d_a, d_b, d_c, NX, NY, NZ
);

CHECK(cudaGetLastError());
CHECK(cudaDeviceSynchronize());

dElaps = cpuSecond() - dStart;

// Copy result back to CPU.
CHECK(cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost));

// TODO 6: Print the last 1000 elements in memory order.
printf("\nLast 1000 results:\n");

for (size_t i = count - 1000; i < count; i++) {

    int z = i / (NX * NY);
    int y = (i / NX) % NY;
    int x = i % NX;

    printf("C[%d][%d][%d] = %d x %d = %d\n",
           z, y, x, a[i], b[i], c[i]);
}

// TODO 7: Verify all elements on the CPU.
int errors = 0;

for (size_t i = 0; i < count; i++) {
    if (c[i] != a[i] * b[i]) {
```

```
                errors++;
            }
        }

        printf("\nVerification: %d mismatches out of %zu\n",
               errors, count);

        printf("Time taken - %.7f\n", dElaps);

        // Free CPU memory.
        free(a);
        free(b);
        free(c);

        // Free GPU memory.
        CHECK(cudaFree(d_a));
        CHECK(cudaFree(d_b));
        CHECK(cudaFree(d_c));

        return 0;
    }
```

```
Writing ex4_volume.cu
```

```
!nvcc -arch=sm_75 ex4_volume.cu -o ex4_volume && ./ex4_volume
```

```
C[499][499][471] = 44 x 62 = 2728
C[499][499][472] = 70 x 56 = 3920
C[499][499][473] = 11 x 80 = 880
C[499][499][474] = 89 x 72 = 6408
C[499][499][475] = 63 x 35 = 2205
C[499][499][476] = 19 x 7 = 133
C[499][499][477] = 71 x 30 = 2130
C[499][499][478] = 53 x 17 = 901
C[499][499][479] = 52 x 19 = 988
C[499][499][480] = 84 x 32 = 2688
C[499][499][481] = 35 x 17 = 595
C[499][499][482] = 8 x 71 = 568
C[499][499][483] = 77 x 9 = 693
C[499][499][484] = 40 x 29 = 1160
C[499][499][485] = 22 x 96 = 2112
C[499][499][486] = 50 x 67 = 3350
C[499][499][487] = 91 x 8 = 728
C[499][499][488] = 88 x 57 = 5016
C[499][499][489] = 53 x 6 = 318
C[499][499][490] = 39 x 68 = 2652
C[499][499][491] = 83 x 84 = 6972
C[499][499][492] = 24 x 11 = 264
C[499][499][493] = 82 x 82 = 6724
C[499][499][494] = 63 x 17 = 1071
C[499][499][495] = 24 x 12 = 288
C[499][499][496] = 10 x 28 = 280
C[499][499][497] = 92 x 74 = 6808
C[499][499][498] = 38 x 62 = 2356
C[499][499][499] = 47 x 68 = 3196

Verification: 0 mismatches out of 125000000
Time taken - 0.0078690
```